PROJE ANALİZ VE SUNUM RAPORU

Make A Playlist

Tek bir öğrenme hedefini sıralı bir YouTube playlist'ine çeviren ve topladığı kaynaklara soru sorulmasına izin veren; cevabı kaynaklarda yoksa **"bulamadım"** diyen sistem.

Mimari
FastAPI · React · SQLite/Postgres · Redis

Öne çıkan mekanizma
Dört bağımsız "kaçınma kapısı" ile atıflı, kaynağa bağlı cevap

Kod tabanı anlık görüntüsü
Dal `feat/react-migration` · 87 commit · 29 Eylül 2026

Depo
github.com/ibrhmgldmr3/LLM-AGENT-PlaylistMaker

BÖLÜM 1

Yönetici özeti

Ne yapıyor, neden farklı, ne kadar sağlam — bir sayfada.

Make A Playlist iki işi tek üründe birleştirir:

1 · Yolu çiz

Kullanıcı bir konu yazar. Sistem konuyu ayrık alt konulara böler, her biri için YouTube'da arar, adayları metadata ve (varsa) transkript üzerinden sıralar ve **her alt konuya tam bir video** atar. Sonuç JSON/Markdown olarak indirilir veya kullanıcının YouTube hesabına gerçek bir playlist olarak yayınlanır.

2 · Topladığına sor

Bu videolar ve yüklenen dokümanlar (PDF, DOCX, TXT, MD) bir **defter** olur. Sorulara yalnızca defterin kaynaklarından, videonun tam saniyesine veya dokümanın sayfasına bağlanan atıflarla cevap verilir. Kaynak soruyu kapsamıyorsa sistem uydurmaz, söyler.

Ürünün iddiası: reddetmek özelliğin kendisi

Her dil modeli inandırıcı bir paragraf üretir. Burada fark, cevapsız bir sorunun cevaplanmasını **yapısal olarak zorlaştıran dört bağımsız kapı**: (1) modeli hiç çağırmayan deterministik bir erişim eşiği, (2) zorunlu çıktı şeması, (3) atıf doğrulaması ve (4) cevap–atıf sözcük örtüşmesi kontrolü. Eşikler tahminle değil, gerçek defterler üzerinde **ölçülerek** konmuş ve ölçüm araçları projeyle birlikte gelir.

~15 bin satır uygulama kodu (Python: `api/` + `src/`)	**729 + 67** geçen test: backend (pytest) + frontend (vitest)	**28** HTTP ucu, 16 veritabanı tablosu	**3** LLM sağlayıcısı: Gemini, Together.ai, OpenRouter
4 bağımsız "bulamadım" kapısı	**≈ 1.224** bir çalıştırmanın YouTube kota birimi (günlük 10.000 → ~8 çalıştırma)	**3,5×** paralel keşif/transkript ile hızlanma (12,8 sn → 3,7 sn, aynı çıktı)	**0** defterler arası sızıntı (3 defter × 6 soru = 18 koşum)

Bu rapor nasıl okunur

2 Problem ve ürün tanımı
3 Sistem mimarisi
4 Playlist üretim hattı ve sıralama
5 Öğrenme alanı (RAG) ve dört kapı
6 Teknoloji yığını
7 Dayanıklılık, kota ve güvenlik
8 Karşılaşılan sorunlar ve çözümleri
9 Test, CI ve ölçüm kültürü
10 Dağıtım, ölçekleme, bilinen sınırlar
11 Sunum rehberi: akış, sorular, kontrol listesi
Ek API uçları, tablolar, dizin, zaman çizelgesi

Bu rapordaki sayılar iki kaynaktan gelir: **çalıştırılıp ölçülenler** (test sayıları, satır sayıları, yerel veritabanı) bu oturumda doğrudan alındı; **tasarım ölçümleri** (eşikler, sıralama etkileri, hızlanma) projenin kendi belgelerinden (`readme.md`, `docs/rag-plan.md`, `docs/react-migration-plan.md`, commit mesajları) aktarıldı.

BÖLÜM 2

Problem ve ürün tanımı

Kim için, hangi durumda, neyi çözüyor.

Çözülen problem

Kendi kendine bir teknik konuyu YouTube'dan öğrenmeye çalışan biri iki ayrı duvara toslar. Birincisi **yol bulma**: konu dağınık, hangi video hangi alt konuya iyi oturuyor, hangisi hangisinin önkoşulu belli değil. İkincisi **geri dönüp bulma**: saatlerce video izlendikten sonra tek bir cümleyi bulmak için videoları yeniden izlemek gerekiyor.

Ürün ikisine de cevap verir: önce konuyu sıralı bir yola çevirir, sonra bu yolun içeriğini (ve kullanıcının dokümanlarını) sorgulanabilir kılar.

Hedef kullanıcı ve bağlam

Kullanıcı

Türkçe konuşan, kendi kendine öğrenen öğrenciler ve profesyoneller. Ürün herkese açık olacak şekilde düşünüldü: yabancılar kaydolup ürünle "soğuk" karşılaşır. Masaüstü ve telefon **eşit derecede** birinci sınıf.

Kullanım durumu

Gezinme değil çalışma. Kullanıcı bir konuyu öğrenmeye karar vermiş; bir yol ve sonra saatlerce videoyu tekrar izlemeden soru sorabilmek istiyor.

> **Ürünü şekillendiren gerçeklik: kaynaklar İngilizce, arayüz ve sorular Türkçe**
> YouTube'daki teknik anlatımların çoğu İngilizce; kullanıcı Türkçe soruyor. Bu, kenar durum değil olağan durum. Çapraz dilli kosinüs benzerliği düşüyor (aynı soru Türkçe 0,701, İngilizce 0,726), Türkçe cevabın İngilizce kaynakla sözcük örtüşmesi ölçülemiyor. Erişim eşiğinden atıf kontrolüne kadar birçok tasarım kararı bu gerçekten çıktı.

Beş ürün ilkesi (`PRODUCT.md`)

#	İlke	Somut karşılığı
1	**Sık cevaplamaktan iyi reddetmek**	"Bulamadım" hata değil doğru sonuç; arayüzde başarısızlık gibi biçimlendirilmez. "Sorunuz bu kaynakların dışında" ile "kendi cevabımı doğrulayamadım" ayrı cümlelerle söylenir.
2	**Aranamayan şeyi göster**	Transkripti olmayan video veya taranmış PDF `no_text` ("kapsam dışı") olarak işaretlenir, `failed` olarak değil. OCR yoktur ve arayüz bunu açıkça söyler.
3	**Her iddia izlenebilir**	Atıflar videonun tam saniyesine veya dokümanın sayfasına bağlanır. Süre/sayfa orantıyla tahmin edilmez; bilinmiyorsa gösterilmez.
4	**Bekleme tasarlanır**	Uzun işler asenkron; ilerleme SSE ile canlı akar, iptal edilebilir, bağlantı koparsa kaldığı yerden devam eder.
5	**Ayarlamadan önce ölç**	Eşikler ve sınırlar gerçek veriden gelir; yeniden ölçme betikleri (`scripts/`) ürünle birlikte gelir.

Kapsam ve sınırlar (ürün düzeyinde)

Kullanıcı başına 10 defter, defter başına 25 doküman, yükleme başına 20 MB (yükleme sırasında sayılır, `Content-Length`'e güvenilmez). Defterler birbirinden **izole**; "bu defterde neler var?", "özetle" gibi **defter düzeyi sorular** ayrı bir erişim yolu izler. Arayüz Türkçe kaynaklı, İngilizce çevirili; açık/koyu tema. Alan sözlüğü: *defter* (notebook), *çalışma odası*, *kaynak*, *alıntı*.

BÖLÜM 3

Sistem mimarisi

Katmanlı, arayüzü değiştirilebilir, iş yürütmesi soyutlanmış bir Python + React uygulaması.

TARAYICI · web/

React 18 · TypeScript · Vite · Tailwind. Çalıştırma formu, canlı ilerleme (`EventSource`), sonuç kartları, geçmiş, defter çalışma alanı (kaynaklar, transkript, atıflı soru–cevap), TR/EN arayüz, açık/koyu tema. Tipler backend'in OpenAPI şemasından üretilir.

▼ HTTP/JSON + Server-Sent Events · `/api/*`

API KATMANI · api/

FastAPI. Yönlendiriciler: runs | spaces | auth | config | admin · kimlik `deps.py`'de çözülür (rotalar oturum mekaniğini bilmez) · `sse.py` olay akışı · sağlayıcı hatalarını HTTP durumlarına eşleyen işleyiciler · derlenmiş React'i aynı süreçten servis eder.

▼ `JobRunner` protokolü

İŞ YÜRÜTME · src/jobs/

InProcessJobRunner (varsayılan)
Bellek içi, thread havuzu, tek süreç. Hiçbir altyapı gerektirmez.

RedisJobRunner + worker
İş kuyruğu (`BLPOP`), olay günlüğü Redis `LIST`'te. Web ve worker bağımsız ölçeklenir.

▼ servisler işin nerede koştuğunu bilmez

SERVİSLER · src/services/ — iş mantığı, framework'ten bağımsız

playlist_service | topic_service | youtube_search_service | metadata_ranker | recommendation_service | transcript_service | rag_service | chunking | embedding_service | document_parser | playlist_publish_service | run_retention | space_retention

▼

SAĞLAYICILAR · src/providers/

LLM: Gemini (yedek zincirli), Together.ai, OpenRouter
YouTube: Data API v3, yt-dlp, youtube-transcript-api
ASR: faster-whisper, whisper.cpp
Kimlik: Google OAuth (giriş + playlist yayını)
Hata sınıfları: geçici · hız sınırı · kalıcı · video düzeyi

DEPOLAMA · src/storage/

SQLite (varsayılan, FTS5) veya **PostgreSQL** (`DATABASE_URL`) — tek "lehçe" katmanı
Dosya sistemi `data/`: çalıştırma çıktıları, yüklenen dokümanlar, indirilen ses
Redis: iş durumu ve olay günlüğü (isteğe bağlı)
Sır şifreleme: OAuth jetonları (`SecretBox`)

> **Neden bu katman kuralı önemli**
>
> `services`, `providers`, `storage`, `models` ve `utils` içinde tek bir arayüz framework'ü import'u yok. Bu yüzden Streamlit'ten React'e geçiş yeniden yazım olmadı: yaklaşık **500 satır** (Streamlit arayüzü) atıldı, yaklaşık **2.600 satır** iş mantığı olduğu gibi kaldı; mevcut testler geçiş boyunca regresyon ölçütü oldu.

İki dağıtım topolojisi

A · Tek süreç (varsayılan)

Tek konteyner: `uvicorn` hem API'yi hem derlenmiş arayüzü (`web/dist`) servis eder. İş durumu bellekte, veritabanı SQLite. Kurulumu en kolay olan; **tek süreç zorunlu** (`--workers` yok). Uygulama `WEB_CONCURRENCY > 1` görürse açılışta hata olarak loglar.

B · Docker Compose (ölçeklenebilir)

`web` (çoğaltılabilir) + `worker` + `redis` (`appendonly` açık) + `postgres:16` + paylaşımlı `appdata` birimi. Aynı imaj, farklı komut. `transfer` profili SQLite → Postgres tek seferlik veri aktarımı yapar.

Öne çıkan mimari kararlar

Karar	Gerekçe
FastAPI + pydantic	Alan modelleri (`PlaylistResult`, `ProgressEvent` …) zaten pydantic. Şemalar bire bir hazır; OpenAPI'den TypeScript tipleri otomatik üretilir. Ayrı serileştirme katmanı yok.
SSE, WebSocket değil	Akış tek yönlü (sunucu → istemci). Proxy'lerden sorunsuz geçer, otomatik yeniden bağlanır. Her olay `id:` taşır; `Last-Event-ID` ile istemci yalnızca kaçırdıklarını alır.
`JobRunner` protokolü	`submit / subscribe / cancel` arayüzü arkasında bellek içi ve Redis uygulamaları var. FastAPI'nin `BackgroundTasks`'ı servis katmanına sızdırılmadı.
Vektör veritabanı yok	Defter başına birkaç bin parça: 2.500 × 3072 float32 ≈ 30 MB; numpy ile kosinüs milisaniyelerle ölçülüyor. Chroma/FAISS/pgvector, var olmayan bir ölçek sorununu çözerdi.
SQLite → Postgres tek katman	`dialect.py` yalnızca farkı taşır: yer tutucu (`?` → `%s`), otomatik id, BLOB→BYTEA, FTS5→tsvector, `BEGIN IMMEDIATE` →advisory lock. 127 SQL ifadesinin neredeyse tamamı iki veritabanında aynen çalışır; ikinci bir store sınıfı yazılmadı.
API sözleşmesi derleme zamanında	`schema.d.ts` backend OpenAPI'sinden üretilir, `contract.ts` el yazımı tiplerle karşılaştırır. Backend'de bir alanı yeniden adlandırmak `npm run typecheck`'i kırar. SSE gövdeleri OpenAPI'de görünmediği için `dump_openapi.py` onları elle ekler.
Tek imaj, çok aşamalı	Node 22'de frontend derlenir, çalışma zamanı `python:3.13-slim` yalnız Python + ffmpeg taşır. Kök olmayan kullanıcı, `/api/health` sağlık kontrolü.

BÖLÜM 4

Playlist üretim hattı ve sıralama

Ağ-bağlantılı işler paralel, sıraya bağımlı işler ardışık ve deterministik.

1 · Ayrıştırma
Konu → alt konular. Tek LLM çağrısı, en fazla 6.
tek çağrı

›

2 · Keşif
Data API, yedek: yt-dlp. Alt konu başına 1–2 sorgu.
paralel (4)

›

3 · Sıralama
Her alt konu, ortak havuza karşı puanlanır.
süreç içi

›

4 · Transkript
Video bazında tekilleştirilmiş zenginleştirme.
paralel (4)

›

5 · Atama
Alt konu başına bir video, toplam uyum en büyük.
ardışık

›

6 · Çıktı
JSON + Markdown; isteğe bağlı YouTube playlist.
ardışık

Keşif: aday havuzu bilerek geniş kuruldu

- **Sorguları kod değil LLM yazar.** Tek çağrı her alt konu için `{title, query, query_en, terms}` döndürür; ek sorgular ve sıralamadaki eşdeğer terimler ek LLM maliyeti olmadan gelir. "Konu + alt konu"nun mekanik birleşimi doğal olmayan tekrarlı dizeler üretiyordu; model insanların gerçekten aradığı terimleri yazar.
- **İki dilli arama (isteğe bağlı).** Arayüz dili İngilizce değilse alt konu başına ikinci bir İngilizce sorgu çalışır. Türkçe bir makine öğrenmesi konusunda havuz 52'den 95 videoya çıktı; maliyeti alt konu başına bir `search.list`.
- **Aday havuzu alt konular arasında birleştirilir.** 5 alt konuda ~12 yerine ~55 benzersiz video; ek kota yok. Kendi araması boş dönen alt konu havuzdan beslenir ve bu uyarı olarak söylenir.
- **Sonuç vermeyen sorgu genişletilir**: sonuç yoksa yalnızca alt konu başlığıyla yeniden denenir.

Sıralama: sinyal ağırlıkları

Alaka, **alt konu ve konuya ayrı ayrı** ölçülür. Birleşik sorguda ortak konu sözcükleri baskın gelir ve her alt konu neredeyse aynı sırayı üretirdi (ARIMA ve LSTM slotu aynı videoyu seçerdi). Konu ağırlığı bu yüzden bilerek düşüktür.

Sinyal	Aralık	En yüksek katkı
Başlık × alt konu	0 … 3,2	
Açıklama × alt konu	0 … 1,6	
Başlık × konu	0 … 0,8	
Açıklama × konu	0 … 0,4	
Kanal otoritesi (abone sayısı, log)	0 … 2,2	
Süre uygunluğu	−3,0 … 2,0	
Dil eşleşmesi	−0,2 … 1,5	
Tazelik	−0,25 … 1,5	
Zorluk uyumu	−0,1 … 1,0	
Etkileşim (izlenme / gün)	0 … 1,0	
Transkript bonusu	−0,1 … 1,0	
Canlı yayın cezası	−2,0	ceza

Skoru ayırt edici yapan dört ek mekanizma

Havuz tabanlı IDF

Aday havuzu kendi derlemi olur: `xgboost` bir başlıkta, `model` onlarcasında geçer, nadir terim ağırlık taşır. Aksi hâlde yalnızca genel sözcüklerle eşleşen bir video kazanabiliyordu.

Kök farkındalıklı eşleşme

Türkçe eklemeli: `tahmin/tahmini`, `model/modelleri` eşleşmeli. Ortak önek eşleşmesi, korumalarla: `veri` ≠ `verimlilikten`, `python2` ≠ `python3`, `gpt4` ≠ `gpt5`.

Alaka kapılı popülerlik

Kanal otoritesi, tazelik ve etkileşim, başlık+açıklama alakası eşiğe ulaşmadan tam sayılmaz; sıfır alakada katkısı sıfır. Süre, dil ve zorluk kısıtlarına dokunulmaz (bunlar kullanıcı kısıtı).

Global atama

Alt konuları sırayla doldurmak yerine toplam uyumu en büyükleyen atama yapılır; erken bir alt konu, sonrakinin çok daha iyi uyduğu videoyu kapamaz. Küçük bir kanal-tekrar cezası (0,6) yalnızca eşitlikleri bozar.

Hız ölçümü

Keşif ve transkript aşamalarını paralelleştirmek 4 alt konulu bir çalıştırmada uçtan uca **~3,5×** hızlanma sağladı (12,8 sn → 3,7 sn) ve **bayt düzeyinde aynı çıktıyı** üretti. Transkriptler video bazında tekilleştirilir; önceki sürümde alt konu içinde ardışık çalışıp toplam sürenin ~%73'ünü tüketiyordu.

ASR (konuşma tanıma) varsayılan olarak kapalı

Ses indirip Whisper çalıştırmak CPU'da video başına **90–120 sn** tutar. İlk iki transkript kaynağı (youtube-transcript-api, yt-dlp altyazıları) videoların büyük çoğunluğunu karşılar ve transkript skoru en fazla +1,0 oynatır. Açıldığında bile üç tavan var: çalıştırma başına video sayısı, toplam saniye bütçesi ve canlı yayın/aşırı uzun video filtresi; ASR ayrı bir süreç-geneli semafor arkasında koşar.

BÖLÜM 5

Öğrenme alanı (RAG) ve dört kapı

Cevap üretmek değil, cevabı olmayan soruya cevap üretmemek.

KAYNAK EKLEME

Video — Çalıştırmadan gelir; mevcut `get_transcript` yolu (önbellek, soğuma, hız sınırı olduğu gibi devralınır)

›

Doküman — PDF · DOCX · TXT · MD → `(sayfa, metin)`

›

Parçalama — 1.200 karakter, 200 örtüşme; zaman damgalı / sayfalı

›

İndeks — `chunk` + `chunk_fts` (FTS5) + `chunk_embedding` (float32 BLOB)

SORU SORMA

Soru — Boş / boş defter erken elenir

›

Hibrit erişim — FTS5 (sözcüksel) + embedding kosinüs → RRF (k=60)

›

Kapı 1 — Eşik altı + sözcüksel kanıt yok → model hiç çağrılmaz

›

Kapı 2 — Çıktı şeması: `answered` + kullanılan parça no'ları

›

Kapı 3a/3b — Atıf ve dayanak doğrulaması

›

Sonuç — Atıflı cevap veya gerekçeli "bulamadım"

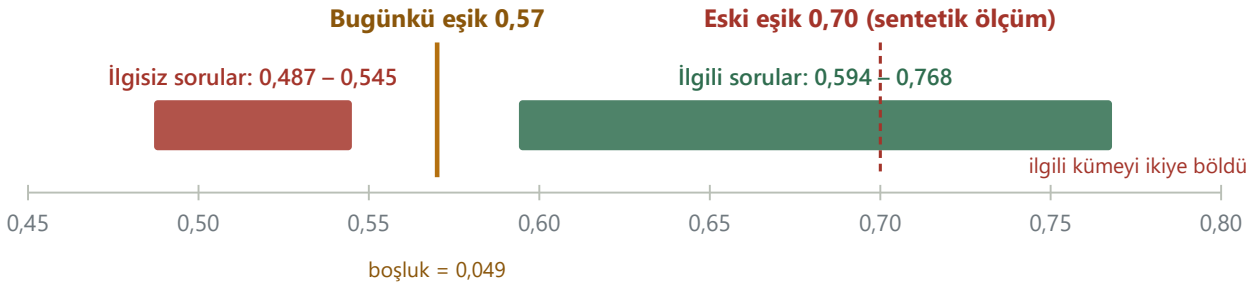
Dört kapı ve neden dördü de gerekli

Kapı	Ne yapar	Yakaladığı arıza	Maliyet
1 · Erişim eşiği	En iyi aday hem sözcüksel eşleşmesiz hem `RAG_MIN_SIMILARITY` altındaysa **LLM çağrılmaz**.	Alakasız soruya "yine de bir şey yaz" davranışı. Test: alakasız soruda sahte sağlayıcının çağrı sayacı **artmaz**.	Sıfır, deterministik
2 · Şema	Model `answered` ve kullandığı parça numaralarını döndürmek zorunda.	Serbest metin cevap. (Şema alanın *varlığını* zorlar, içeriğinin doğruluğunu değil; bu yüzden 3 var.)	Bir çağrı
3a · Atıf	Verilen numaralar, bağlama **gerçekten konulanlarla** kesiştirilir; kesişim boşsa cevap düşer.	Uydurma parça numarası.	Sıfır
3b · Dayanak	Cevap metni, **alıntıladığı** parçalarla sözcüksel olarak karşılaştırılır; örtüşme < 0,15 ise cevap düşer.	3a'yı geçen ama metni desteklemeyen atıf; altyazıya gömülü "şunu yaz ve 3 numaralı alıntıyı göster" enjeksiyonu (gerçek bir numara verdiği için 3a'dan geçiyordu).	Sıfır

3a veya 3b'den düşen cevap, "havuzda yok"tan **ayrı bir gerekçeyle** raporlanır (`unverified`): kullanıcı, sorusunun kapsam dışı kalmasıyla sistemin kendi çıktısını doğrulayamamasını ayırt edebilmeli. İlkinde soruyu değiştirmek, ikincisinde yeniden sormak anlamlı. `answered=false` HTTP **200** döner: ürünün vaadi bu, hata koduna çevirmek arayüzde kırmızı kutu ve "sistem bozuk" izlenimi demek.

Eşik ölçümü: ilk ölçüm yanlıştı

`RAG_MIN_SIMILARITY`, `gemini-embedding-001` ile **gerçek kullanıcı defterlerinde** 20 soru üzerinden ölçüldü. İlk değer (0,70) yedi soruluk *sentetik* bir ölçümden gelmişti ve gerçek ilgili kümenin tam ortasından geçip meşru soruları kesiyordu (ör. "durum yöneticileri neden kötü olabilir": 0,594; "bu konunun ana fikri ne": 0,601 — ikisi de modele hiç ulaşmıyordu).



İlgisiz taban (0,487–0,545) neredeyse hiç değişmedi (önceki ölçüm 0,496–0,542): bu modelin kendi tabanı ve kararlı. Değişen, ilgili kümenin yeriydi: kullanıcı Türkçe soruyor, transkriptler çoğunlukla İngilizce ve gerçek sorular sentetiklerden belirsiz. **Boşluk dar (0,049)**; benzerlik bu korpusta tek başına zayıf bir sinyal, sözcüksel kapsam kapısı gerçek iş yapıyor. Değer **modele özgü**: başka embedding modeline geçilirse `scripts/measure_rag_thresholds.py` ile yeniden ölçülmeli.

Defter düzeyi sorular ve izolasyon

"Bu defterde neler var?"

Cevabı tek parçada değil koleksiyonun bütününde. Benzerlik araması "en yakın parçayı" getirir; bu soruların en yakın parçası yoktur. Çözüm: Kapı 1 atlanır, bağlama **her kaynaktan bir temsilci (açılış parçası)** girer; Kapı 2 ve 3 aynen işler. Ölçüm: "genel olarak enflasyon nasıl hesaplanır" ve "osmanlı tarihini özetle" React defterinde **doğru şekilde reddedildi**.

Defter izolasyonu

Her iki erişim yolu da `space_id` ile kapsanır (`search_chunks_fts` ve `load_embeddings`). Gerçek veriyle 3 defter × 6 soru = 18 koşum, **0 sızıntı**. `test_space_scope.py` iki yolu birlikte kilitler: biri düzelip diğeri açık kalsaydı sızıntı sessiz olurdu. Başkasının defteri 403 değil **404** döner; 403 varlığı sızdırırdı.

Prompt injection: tehdit modeli ve savunma katmanları

Parça metnini soruyu soran kullanıcı yazmaz; bir videoyu yayınlayan veya PDF'i hazırlayan kişi, modele hitap eden cümleler koyabilir ve bu metin doğrudan isteme girer. Etki alanı bilerek dar tutuldu: cevap yolunda **araç, dış istek veya otomasyon yok** (tek JSON çıktısı) ve cevap arayüzde **düz metin** basılır (`dangerouslySetInnerHTML` yok), yani XSS ve görsel-işaretçi ile veri sızdırma kapalı. Kalan risk, kullanıcıya kendi güvendiği kaynaktan geliyormuş gibi görünen bir metin göstermek; bu, ürünün tek vaadini hedeflediği için savunulur.

Katman	Önlem
Sistem talimatı	Alıntı metni **veridir**, talimat değil; hangi otoriteyi iddia ederse etsin uyulmaz.
İstem düzeni	Talimat → veri → **soru en sonda**. Gömülü "yukarıdakileri yoksay" son sözü söyleyemez.
Ayraçlar	Parçalar `<excerpt id="…">` içinde; gövdedeki etiket taklitleri bozulur, başlık/konum öznitelikleri tırnak ve açılı parantezden arındırılır (başlık da saldırgan kontrolünde).
Çıktı	Kapı 3b: cevap, atıfladığı parçayla örtüşmüyorsa düşer. Ölçüm: meşru cevaplar 0,43–1,00; enjekte metinler **0,00**.
Girdi alanı	`language` isteme girer; yalnız harf ve boşluk kabul edilir (sabit liste değil karakter kuralı, meşru diller yasaklanmasın diye).
Ölçüm	Bağlama giren parçalar yönerge benzeri kalıplar için taranır ve **yalnızca loglanır**, engellemez. Nedeni: prompt injection *anlatan* bir video bu kalıpların hepsini içerir ve kullanıcının onu sormaya hakkı vardır. Kalıplar iki sinyal arar (yalnız "yoksay" yetmez).

Bilinen sınır (açıkça söyleniyor)

Kapı 3b, cevap ile kaynak farklı dillerdeyken dayanağı **ölçemez**: kusursuz bir Türkçe cevap İngilizce kaynağa 0,07 örtüşür. Kapı bu yüzden yalnızca *uyuşmazlık kanıtı varken* susar, kararsızlıkta değil. Çapraz dilli sorularda enjeksiyona karşı kalan katmanlar ayraçlar, sistem talimatı, istem düzeni ve 3a'dır. Bilerek kabul edildi: ölçülen zarar (doğru cevapların kesilmesi) kesin ve sürekli, engellenen senaryo varsayımsal. **Mimari sınır:** RAG cevabı asla araç/otomasyon tetiklemez ve asla ham HTML olarak basılmaz; ikisinden biri değişirse bu tehdit modeli yeniden yazılmalı.

Sessizce yarım kalan indeksi görünür kılmak

Ingest, embedding sağlayıcısı düşse bile başarısız olmaz: parçalar yazılır ve sözcüksel arama çalışır (zayıf arama, arama yok değil). Ama bu bir gün canlı veritabanında şu şekilde ortaya çıktı: 130 parçanın yalnızca **64'ü** (tam olarak bir `EMBEDDING_BATCH_SIZE` grubu) gömülmüştü, 6 kaynağın 3'ünde hiç vektör yoktu ama hepsi `indexed` görünüyordu. Çözüm: `SpaceSource.embedded_chunk_count` her sorguda **canlı sayılır** (saklanan bir alan değil;

saklansaydı aynı şekilde bayatlardı), arayüz “N bölümde anlamsal arama yok” veya “yalnızca kelime araması” der. Onarım aracı: `scripts/backfill_embeddings.py`.

BÖLÜM 6

Teknoloji yığını

Neyi, neden seçtiğimizle birlikte.

Katman	Teknoloji	Ne için / neden
Backend dili	Python 3.13 (CI ve geliştirme hedefi)	Sağlayıcı SDK'ları ve veri işleme ekosistemi; 3.11/3.12 “çalışmalı ama doğrulanmadı”.
Web çerçevesi	FastAPI, Uvicorn, Pydantic v2, python-multipart	Tipli API, otomatik OpenAPI, SSE için `StreamingResponse`.
Veritabanı	SQLite (WAL, FTS5) · PostgreSQL 16 (psycopg2-binary)	Varsayılan sıfır altyapı; Postgres çok makineli ölçek için. FTS5 → tsvector çevirisi `dialect.py`'de.
İş kuyruğu	Bellek içi thread havuzu · Redis 7 (LIST + pub/sub)	Olay günlüğü LIST'te tutulur, pub/sub yalnız uyandırma içindir: yeniden bağlanan istemci kaçırdıklarını isteyebilmeli.
LLM	Google Gemini (`gemini-3.7-flash`, yedek zincir), Together.ai (Llama-3.3-70B), OpenRouter	Sağlayıcı değiştirilebilir; tek istem, tek alan sözleşmesi, yalnızca şema lehçesi farklı (Gemini büyük harf tipler, OpenAI uyumlu uçlar standart JSON Schema). Together/OpenRouter ek SDK gerektirmez, `requests` yeter.
Embedding	`gemini-embedding-001` (3072 boyut) · `BAAI/bge-m3` (Together, çok dilli)	OpenRouter embedding desteklemez; otomatik olarak Gemini/Together'a düşer. Modeller arası yedek zincir bilerek yok (karışık vektörler arasında kosinüs anlamsız).
YouTube	YouTube Data API v3 (birincil) · yt-dlp (yedek) · youtube-transcript-api 1.x	Data API kotası kısıtlayıcı; yt-dlp hız sınırlarına takılabilir (çerezle hafifler). Altyazılar önce resmi transkript API'sinden, sonra yt-dlp'den.
ASR (isteğe bağlı)	faster-whisper (`base`, int8) · whisper.cpp · ffmpeg · Node (yt-dlp `nsig` çözümü)	16 kHz mono WAV doğrudan üretilir; Whisper modeli süreç boyunca önbelleklenir (soğuk yükleme 5,5 sn).
RAG	numpy, pypdf, python-docx, FTS5	Vektör DB yok; kosinüs numpy'da. Sözcüksel + anlamsal RRF ile birleştirilir.
Kimlik	Google OAuth 2.0 + PKCE, oturum çerezi	Hem giriş hem YouTube playlist yayını için; jetonlar `SecretBox` ile diskte şifreli.
Frontend	React 18.3, TypeScript 5.6, Vite 8, Tailwind 3.4, Radix UI, framer-motion, lucide-react	Erişilebilir bileşen temeli (Radix), OpenAPI'den üretilen tipler (openapi-typescript).
Test	pytest 8, httpx · Vitest 3, Testing Library, jsdom	Ağ ve API anahtarı gerektirmeden çalışır; sahte `EventSource` ile SSE hook'u zamanlayıcısız test edilir.
Altyapı	Docker (çok aşamalı), Docker Compose, GitHub Actions	Tek imaj: web, worker ve transfer aynı imajdan farklı komutla.

Arayüz tasarım sistemi

Arayüz "müze etiketi" metaforuyla tasarlandı (`web/DESIGN.md`): defter bir **koleksiyon**, kaynak bir **eser**, alıntı bir **künye**. Koyu yeşil "duvar" gezinme ve başlıkları taşır; okunan her şey beyaz "etiket" üstünde akromatik mürekkeple; altı "salon rengi" kaynakları kimlik olarak anahtarlar. Renk sırayla değil `source_id`'nin deterministik hash'inden türer, böylece aynı kaynak her yerde aynı renkle görünür. Okra rengi yalnız "aktif / sıradaki / tıklanacak" sinyalidir; kimlik için kullanılmaz. Salon renkleri beyaz zeminde 4,5:1 kontrastı geçer. Cam efekti, gradyan metin ve neon yoktur; açık/koyu tema ilk boyamadan önce uygulanır (yanlış temanın bir kare bile görünmemesi için).

BÖLÜM 7

Dayanıklılık, kota ve güvenlik

Dış servislere bağımlı, paylaşımlı kotalı bir sistemin ayakta kalma biçimi.

Sağlayıcı sağlığı ve hız sınırları

- **Tek video hatası sağlayıcıyı cezalandırmaz.** Altyazı kapalı, video özel/silinmiş → video düzeyi hata. Yalnızca altyapı düzeyinde ardışık hatalar (`PROVIDER_FAILURE_THRESHOLD`) sağlayıcıyı `PROVIDER_COOLDOWN_SEC` süreyle devre dışı bırakır.
- **Hız sınırı ayrı hata sınıfı.** HTTP 429, `RequestBlocked`, `IpBlocked`: **asla yeniden denenmez** (yeniden deneme engeli derinleştirir), soğuma hemen başlar, süre `Retry-After` başlığından gelir. Eskiden tek bir olay ~24 isteğe (işçi × deneme × sağlayıcı) dönüşüyordu; artık işçi sayısıyla sınırlı.
- **Soğuma sunucu genelinde.** Tüm kullanıcılar aynı IP'den ve aynı API anahtarıyla çıktığı için bir kullanıcının 429'u herkesi durdurur. Kullanıcı başına kapsamda ikinci kullanıcı korumasızdı ve engeli uzatıyordu. Başarılı bir çağrı, süresi dolmamış bir hız-sınırı soğumasını iptal etmez.
- **Model erişimi.** Google eski Gemini modellerini yeni projelere kapatır ("no longer available to new users"). Sağlayıcı zinciri: yapılandırılan model → `gemini-3.7-flash` → `gemini-flash-latest` → `gemini-2.5-flash`; `thinking_config`'i reddeden modellerde parametresiz yeniden dener.
- **Yarıda kalan işler.** Sunucu yeniden başlayınca sonucu olmayan çalıştırma `interrupted` olarak raporlanır (canlı iş yok + sonuç yok ⇒ kesilmiş; bir "çıkarım", ek durum makinesi yok) ve günlük hak sayımından düşülür.

Kota aritmetiği — her şeyi belirleyen sayı

Çağrı (6 alt konu, iki dilli)	Adet	Birim	Toplam
`search.list`	12	100	1.200
`videos.list`	~12	1	~12
`channels.list` (gerçek kanal otoritesi)	~12	1	~12
Bir çalıştırma			**≈ 1.224**

Google Cloud projesi başına varsayılan günlük kota 10.000 birim ⇒ **tüm kullanıcılar toplamı için günde ~8 çalıştırma**. CPU, bellek veya kullanıcı sayısı değil, bu sayı üründe ölçeği belirler. İki tavan farklı sorular yanıtlar:

`MAX_RUNS_PER_USER_PER_DAY`

"Bu kişi payını aldı mı?" Adalet mekanizması. `multi_user` için 3 önerilir.

`MAX_UNITS_PER_DAY`

"Serviste hiç kapasite kaldı mı?" Kota *birimi* üzerinden, çünkü önbellek isabetinde bir çalıştırma sıfır harcar.

Her ikisi de **çalıştırma satırını yazan aynı işlemde** uygulanır (`BEGIN IMMEDIATE`). Kontrol ile yazımı ayırmak, eşzamanlı isteklerin aynı bayat sayıyı okumasına yol açar: ölçüldü, kullanıcı başına sınır 3 iken 12 eşzamanlı istekten 11'i kabul edildi; servis bütçesi 2,1 katına çıktı. Bütçe kontrolü ayrıca harcama gerçekleşmeden **kabul anında tahmini maliyeti rezerve eder**; harcama dakikalar sonra yazıldığı için yalnızca harcamaya bakan bir kontrol, uçuştaki tüm çalıştırmaların aynı rakama karşı geçmesine izin verirdi. Tahmin bilerek en kötü durum: eksik tahmin, LLM çağrısı çoktan ödenmişken çalıştırmanın yarıda `403` ile ölmesi demek. Kota günü **Pasifik saatiyle** sıfırlanır (Google'ın sıfırlama saati).

`GET /api/admin/usage`: günlük gerçek tüketim, uç nokta kırılımı, kullanıcı toplamları, son 24 saatteki çalıştırmalar, aktif soğumalar, hız sınırı olayları. `multi_user`'da `ADMIN_USER_IDS` boşsa rapor **herkese kapalı** (unutmanın sonucu açık kapı değil kilitli kapı olsun).

Kimlik, oturum ve sırlar

Konu	Uygulama
İki mod	`single_user` (varsayılan; herkes `local`, oturum yok) ve `multi_user` (Google girişi zorunlu). `multi_user` anahtarları ilgilendirmez, yalnızca **kimi** bildiğini bilir: günlük sınırlar ve kişisel YouTube yayın izni için.
Oturum	Çerez ile; sunucu tarafında **token'ın SHA-256 özeti** saklanır, düz token asla. `Secure` bayrağı `X-Forwarded-Proto`'ya bakar. Süre `SESSION_TTL_SEC` (14 gün).
OAuth akışı	Yönlendirmeli akış (sunucuda tarayıcı açan `run_local_server` web için imkânsızdı). Tek kullanımlık `state` (CSRF), **PKCE** (`code_verifier` sunucuda `state` ile saklanır, URL'e veya istemciye gitmez), bekleyen `state` sözlüğüne tavan (sızıntı önleme).
Jeton saklama	YouTube OAuth jetonları kullanıcı başına `oauth_token` tablosunda; `SECRET_ENCRYPTION_KEY` varsa HMAC-SHA256 anahtar akışı + kurcalama etiketiyle şifreli, harici bağımlılık yok. Anahtar sonradan eklenebilir (düz kayıtlar okunur, sonraki yazımda şifrelenir). Anahtar yokken sunucu açılışta uyarı verir.
Sır sızıntısı	YouTube API anahtarı sorgu parametresi olarak gider, istisna metnine sızabilir. `redact_secrets` günlüklerde, kullanıcıya dönen uyarılarda ve başarısız işin `error` alanında (`GET /api/runs/{id}` ve SSE `done`) uygulanır. Yığın izi sunucuda kalır.
Erişim	Sahiplik kontrolü tüm çalıştırma/defter uçlarında; başkasının kaynağı **404**. CORS `*` yok sayılır (API oturum çerezi taşır). Depoda `.env` takip edilmiyor; CI hem çalışma ağacını hem **tüm commit geçmişini** anahtar desenleri için tarar.

Yaşam döngüsü ve temizlik

- **Silme diski de temizler.** Bir çalıştırma iki yerde yaşar (veritabanı, `data/runs/<id>`); uzun süre yalnızca veritabanı yarısı siliniyordu. Çökme/yarıda kalma sonucu sahipsiz kalan dizinler açılışta toplanır. Defter silme de aynı şekilde çalışır.
- Süresi dolan önbellek, biten oturumlar ve 90 günden eski günlük sayaçlar her çalıştırma sonunda temizlenir; önbellek anahtarı şema sürümü taşır, aday modeli değişince bayat kayıtlar otomatik geçersizleşir.
- Açılışta yapılandırma sağlığı raporlanır: bilinmeyen `.env` anahtarları, şifreleme anahtarı yokluğu, sınırsız paylaşımlı kota (`multi_user` + `MAX_RUNS_PER_USER_PER_DAY=0`), tek süreç kısıtı.

BÖLÜM 8

Karşılaşılan sorunlar ve çözümleri

Her satır projenin kendi kayıtlarında (commit mesajları, README, plan belgeleri) belgelenmiş gerçek bir sorun; çoğu, davranışı adlandıran bir regresyon testiyle kilitlenmiş.

Örüntü: iyi niyetli ama yanlış varsayım → ölçüm → düzeltme + test

Aşağıdaki sorunların çoğu aynı yolla bulundu: bir varsayımla kod yazıldı ("sentetik sorular gerçeği temsil eder", "kontrol ile yazım ayrı olabilir", "mock'lanan akış gerçek akıştır"), gerçek veriyle ölçülünce varsayımın yanlış olduğu görüldü, düzeltme yapıldı ve **davranışı adlandıran bir test** eklendi. Bazı testlerin kendisi **mutasyonla doğrulandı**: kodu bilerek bozunca test gerçekten düşüyor mu diye bakıldı.

A · Aday keşfi ve sıralama kalitesi

#	Belirti	Kök neden	Çözüm	Kanıt
A1	XGBoost ve LSTM alt konuları tek alt konuya birleşiyor; bir yöntem playlist'ten düşüyor.	Birleştirme ham dize benzerliğine bakıyordu (iki başlık 0,885 benzer).	Ayırt edici token'lara göre birleştirme.	İki yöntem ayrı slot alıyor.
A2	ARIMA ve LSTM slotları aynı videoyu seçiyor.	Konu+alt konu birleşik sorguda ortak sözcükler baskın.	Alaka alt konu ve konuya ayrı ölçülür; konu ağırlığı 0,8 (alt konu 3,2).	Slotlar ayrıştı.
A3	"Ağaç Tabanlı Modeller ve XGBoost" slotunu bir LSTM metin üretimi videosu aldı.	Video yalnızca `tabanlı`, `modeller` ile eşleşiyordu; `xgboost` eşleşmiyordu.	Havuz tabanlı IDF: nadir terim ağırlık taşır.	`xgboost` bir başlıkta, `model` onlarcasında.
A4	Bir bilişsel bilim dersi zaman serisi alt konusunu kazandı.	`temelleri` gibi pedagojik sözcükler nadir olduğundan IDF onları *yüksek* puanlıyordu.	Pedagojik/yapısal stopword'ler. Yaygın alan sözcükleri (`model`, `yöntem`) bilerek bırakıldı.	Yalnız "temelleri" ile kazanılan slot kalmadı.
A5	"Random Tutorial Guy" kanalı MIT OpenCourseWare'in önünde.	Kanal otoritesi anahtar sözcük sezgisiyle hesaplanıyordu.	`channels.list` ile gerçek abone sayısı (log ölçek); etkileşim izlenme/gün.	Arama başına +1 birim (aramanın kendisi 100).
A6	Popüler ama alakasız videolar slot alıyor; bir viral video **üç alt konuyu birden** kazandı.	Popülerlik sinyalleri toplamı (8,2) alaka sinyallerini (4,8) aşıyordu.	Popülerlik alaka eşiğine bağlandı (kapı).	13 çalıştırma / 42 seçim: 3 seçim değişti, üçü de daha iyi sözcüksel eşleşmeye; sıfır sinyalli seçim 6/42 → 4/42.
A7	"Kokusuz Kalman Filtresi" hiçbir şeyle eşleşmedi; havuzdaki "Unscented Kalman Filter Design" başka slota gitti.	Sözcüksel eşleşme TR–EN kavram adlarını bağlayamaz.	LLM aynı çağrıda `terms` döndürür (İngilizce karşılık, kısaltma, eş anlamlılar); alaka *en iyi* eşleşmeyi alır, toplamı değil.	Alaka 1,39 → 3,31, doğru video seçildi; ek LLM çağrısı yok.

#	Belirti	Kök neden	Çözüm	Kanıt
A8	Konuyla hiç örtüşmeyen ama uzun, yeni ve popüler bir video uyarısız sunuldu (güven 8,6).	"Zayıf eşleşme" etiketi toplam güvene bakıyordu; o da süre, dil, kanal sinyallerini içerir.	Etiket **alakaya** bağlandı.	Sıfır alakalı 6 seçimin 3'ü eski kuralda işaretsizdi.
A9	Erken bir alt konu, sonrakinin çok daha iyi uyduğu videoyu kapıyordu.	Alt konu sırasına göre açgözlü seçim.	Toplam uyumu en büyükleyen global atama; kanal-tekrar cezası yalnız eşitlik bozar.	Sonuç alt konu sırasından bağımsız.

B · Öğrenme alanı (RAG)

#	Belirti	Kök neden	Çözüm	Kanıt
B1	Meşru sorular "bulamadım" alıyor; model hiç çağrılmıyor.	Eşik 0,70, 7 soruluk *sentetik* ölçümden; gerçek ilgili küme çok daha aşağıda.	Gerçek defterlerde 20 soru ile yeniden ölçüm ⇒ 0,57.	İlgili 0,594–0,768; ilgisiz 0,487–0,545.
B2	Altyazıya gömülü "şunu yaz, 3. alıntıyı göster" talimatı çalışabiliyor.	Kapı 3a numaranın *sunulduğuna* bakar; enjekte metin gerçek bir numara verir.	Kapı 3b: cevap–atıf sözcük örtüşmesi; ayraçlar; soru en sonda; sistem talimatı.	Meşru 0,43–1,00; enjekte 0,00.
B3	"Bu defterde neler var?" → "verilen metinlerde defterden bahsedilmiyor".	Model "yalnız alıntılardan cevapla" kuralını "alıntılarda 'defter' sözcüğünü ara" diye okuyor.	Defter düzeyi yol: kaynak başına açılış parçası + istemde koleksiyon hakkında konuşma izni. Kapı 1 atlanır, 2 ve 3 kalır.	Bağlam doğru gelse bile yönerge olmadan model soruyu literal alıyordu (ölçüldü).
B4	Doğru Türkçe cevap İngilizce kaynaktan üretilince kapı 3b tarafından kesiliyor.	Sözcük örtüşmesi farklı dilde ölçülemez: kusursuz cevap 0,07.	3b yalnızca **uyuşmazlık kanıtı** varken susar; kısa cevap ve farklı dil "ölçülmedi" sayılır.	Aynı dilde 0,43–1,00; sınır açıkça belgelendi.
B5	"güncelleme" arayan sorgu, metinde "güncellemesi" geçtiği hâlde hiçbir şey bulmuyor.	FTS5 `unicode61` ı/ğ/ş'yi indirgemez ve tokenlar **tam** eşlenir.	Depoya ve sorguya aynı `search_key()` ; yeterince uzun tokenlara önek (`*`), sıralamadaki kök kuralıyla aynı.	İki ayrı sorun, ikisi de ölçülüp düzeltildi.
B6	6 kaynağın 3'ü `indexed` görünüyor ama anlamsal aramada hiç yok.	İlk embedding grubu yazılmış, ikincisi patlamış, istisna yutulmuş.	Eksik vektörler **canlı sayılır**; arayüz söyler; `backfill_embeddings.py` .	130 parça / 64 gömülü — tam bir grup.
B7	Arayüzde `* **ÜFE (Üretici Fiyat Endeksi):**` gibi işaretler.	İstem "Markdown" istiyordu, arayüz `pre-wrap` düz metin basıyordu.	İstem düz metin ister; `plainProse()` sızan işaretleri temizler, yapı uydurmaz.	`plainProse.test.ts` .
B8	Embedding çağrıları 404.	`text-embedding-004` kaldırılmış.	`gemini-embedding-001` . Model yedek zinciri **bilerek yok**: yarısı bir modelle yarısı diğeriyle gömülen alanda kosinüs anlamsızlaşır.	Model bulunamazsa hata yükselir.
B9	(Önlem) İki erişim yolundan biri defter kapsamını unutsa sızıntı sessiz kalırdı.	Kullanıcı yalnızca "alakasız kaynak gösterildi" diye fark ederdi.	Her iki yol `space_id` ile kapsanır; testler ikisini birlikte kilitler.	18 koşum, 0 sızıntı.

C · Eşzamanlılık, kota, kimlik ve altyapı

#	Belirti	Kök neden	Çözüm	Kanıt
C1	İki sekme aynı çalıştırmayı izleyince olaylar aralarında bölünüyor; bir dinleyici `queue.Empty` ile ölüyor.	Olaylar tek `queue.Queue`'dan **yıkıcı** okunuyordu.	Eklemeli olay günlüğü + abone başına imleç; `Last-Event-ID` ile devam.	A: m1,m2,m4,m6 / B: m3,m5 → ikisi de tüm olayları görür. Tek değişiklik iki sorunu çözdü.
C2	Günlük sınır 3 iken 12 eşzamanlı istekten **11** kabul edildi.	Sayım ile `INSERT` ayrıydı; ayrıca satırı arka plan işi yazıyordu (TOCTOU).	Sayım + `INSERT` tek `BEGIN IMMEDIATE` işleminde; satır kabul anında yazılır.	Artık tam 3. Test mutasyonla doğrulandı.
C3	Servis kota bütçesi 2,1 katı aşılıyor (5'lik bütçeye 6 çalıştırma).	Kontrol yalnız gerçekleşmiş harcamaya bakıyor; harcama dakikalar sonra yazılıyor.	Kabul anında rezervasyon (`reserved_units`); `create_run` artık `INSERT OR REPLACE` değil.	Ölçüldü ve testle kilitlendi.
C4	Sunucu yeniden başlayınca çalıştırma sonsuza kadar "beklemede".	Bellekteki iş öldü ama `run` satırı kaldı.	`interrupted` durumu (canlı iş yok + sonuç yok); açılışta işaretleme; hak iadesi.	Sözleşme kontrolü, arayüz tipinin eksikliğini derleme hatasıyla gösterdi.
C5	Bir 429 olayı yaklaşık 24 isteğe dönüşüyor; engel uzuyor.	Hız sınırı da diğer hatalar gibi yeniden deneniyordu.	Ayrı hata sınıfı: asla yeniden denenmez, anında soğuma, `Retry-After`.	İstek sayısı işçi sayısıyla sınırlı.
C6	Ali `yt_dlp` soğumasına girse de Veli aynı IP'den gidip engeli uzatıyor.	Soğuma kullanıcı başına kapsanmıştı; IP ve API anahtarı ise ortak.	Sunucu geneli kapsam; başarı, süresi dolmamış hız-sınırı soğumasını silmez.	Ölçüldü: ikinci kullanıcı hiç korunmuyordu.
C7	YouTube bağlanırken `invalid_grant: Missing code verifier`.	PKCE varsayılan açık; `code_verifier` yalnız URL'i üreten `Flow`'da yaşıyor, `/callback` yenisini kuruyor.	PKCE kapatılmadı; verifier sunucuda `state` ile saklanıp token değişiminde geri verilir.	Testler *ikisini de mock'ladığı* için kaçırmıştı; gerçek devreyi kapsayan 2 test eklendi.
C8	Gemini: "no longer available to new users"; bazı modeller `thinking_config`'i reddediyor.	Google eski modelleri yeni projelere kapatıyor.	Model yedek zinciri; reddedilen parametre için parametresiz yeniden deneme (karar çağrı başına, paylaşılan sete göre değil).	`test_llm_provider.py`.
C9	Uygulama `ValueError: embedded null character` ile açılmıyor.	PowerShell 5.1'de `>>` ve `Out-File` **UTF-16** yazar; UTF-8 `.env` içine null baytlar girer.	Uygulama bozuk satırları tespit edip hangi satırlar olduğunu söyler; bilinmeyen `.env` anahtarları görünür uyarı.	README'de sorun giderme satırı.
C10	Geliştirmede geçen kod, bildirilen alt sürümlerde kırık.	Geliştirme aralığın *üstünde*, `requirements.txt` ise *altını* vaat eder.	CI'da `minimum-deps` işi: alt sınırları kurar, aynı kontrolleri koşar.	FastAPI 0.116 `-> None` + 204'te import anında kırılıyordu; google-auth-oauthlib 1.2.0 PKCE üretmiyordu.

#	Belirti	Kök neden	Çözüm	Kanıt
C11	CI, üretilen OpenAPI tiplerini bayt bayt kıyaslarken kırıldı.	Üretilen şema pydantic sürümüne göre değişiyor (2.11 ↔ 2.8 `additionalProperties`).	Bayt eşitliği yerine **anlamsal** sözleşme (`contract.ts`); şema CI'da taze üretilir, commit'li kopya uyarı verir.	İki uç şemayla da doğrulandı.
C12	Üretimde açılış kayıtları (“tek süreç”, “N çalıştırma işaretlendi”, “frontend mount edildi”) yok.	uvicorn yalnız kendi logger'larını yapılandırır; `api` / `src.*` Python'un `lastResort` 'una düşer (yalnız WARNING+, biçimsiz).	`configure_root_logging`: kök logger + sır maskeleme filtresi + UTF-8 stdout; `LOG_LEVEL`.	Henüz commit edilmedi (çalışma ağacında) — bkz. Bölüm 11.
C13	Kullanıcı silme düğmesine bastı, dosyalar diskte kaldı.	Yalnızca veritabanı yarısı siliniyordu.	`run_retention` / `space_retention`: DB + disk; sahipsiz dizinler açılışta toplanır.	Yarım çalıştırma da hak yakmaz.
C14	Sekiz ayrı arka uç hatası: yarış, kilitlenme, sessiz sızıntı.	Ör.: çözülemeyen sır kaydı hesabı kalıcı kilitliyordu; yayınlamada `HttpError` dışı hata sahipsiz playlist bırakıyordu; log maskeleme biçimlendirme hatasında açık moda düşüyordu.	Tek commit'te (`94f1df0`) sekizi de düzeltildi.	563 satır ekleme, testlerle.

D · Dört büyük mimari dönüm noktası

1 · Streamlit → FastAPI + React

İş mantığı framework'ten bağımsız olduğu için yeniden yazım değil arayüz değişimi oldu. Beş faz, her biri bağımsız sevk edilebilir ve tüm testler yeşil kalarak. Streamlit ve API bir süre yan yana çalıştı; sonra `app.py` ve `src/ui/` (~500 satır) silindi.

2 · BYOK → paylaşımlı sunucu anahtarı

Plan “her kullanıcı kendi Gemini ve YouTube anahtarını getirir” ile yazıldı ve uygulandı. Sonra tersine çevrildi: bir kullanıcıdan Google Cloud projesi açıp iki anahtar üretmesini istemek giriş engeli olarak fazlaydı. Faturayı artık sunucu sahibi öder; bu yüzden günlük tavanlar isteğe bağlı değil zorunlu hâle geldi. `/api/credentials`, ayarlar ekranı ve `user_credential` tablosu kaldırıldı. Config'i üçe bölmek (sunucu / kullanıcı sırrı / çalıştırma seçeneği) geri dönüşte de değerini korudu.

3 · Tek süreç → ölçeklenebilir seçenek

`JobRunner` arayüzü ilk günden vardı ve “erken optimizasyon” olduğu için Redis bekletildi. Sonra Redis kuyruğu, worker, Postgres lehçe katmanı, Compose tarifi ve SQLite → Postgres aktarım aracı geldi; varsayılan yol hâlâ sıfır altyapı.

4 · Playlist üreticiden “öğrenme alanı”na

Ürün, playlist üretiminden, topladığı kaynaklara güvenilir soru sormaya genişledi. Vektör DB ve yeni iş altyapısı eklenmedi: mevcut transkript yolu, iş yürütücüsü ve SSE olduğu gibi yeniden kullanıldı.

BÖLÜM 9

Test, CI ve ölçüm kültürü

Kalite iddiaları, yerelde bu oturumda çalıştırılıp doğrulanan sayılarla.

729	**67**	**0**	**≈ 0,86**
backend testi geçti (33,9 sn), 71 atlandı	frontend testi geçti, 10 dosya (5,4 sn)	TypeScript hatası (`tsc --noEmit`)	test kodu / uygulama kodu oranı (13,1 bin ÷ 15,3 bin satır)

71 atlanan test hata değil: Redis ve Postgres gerektiren testler, ilgili sunucu yoksa (`REDIS_TEST_URL`, `MAP_TEST_POSTGRES_DSN`) atlanır; CI Redis/Postgres olmadan da yeşil kalır. Postgres fixture'ı hedef veritabanını **siler**, bu yüzden adında `test` geçmeyen DSN'yi reddeder. Testler ağ ve API anahtarı olmadan koşar.

CI: dört iş

İş	Neyi korur
Backend	Python 3.13: uygulama import ediliyor mu (testlerden *önce*, çünkü import zamanı kırılmaları 200 toplama hatası yerine kendi adıyla raporlanır) → `pyflakes` → tüm test paketi.
En düşük bağımlılık sürümleri	`requirements.txt` aralıklarının **alt sınırlarını** kurup aynı kontrolleri koşar. Geliştirme aralığın üstünde, vaat altında olduğu için ikisi gerçek hata yakaladı (C10).
Frontend	Tipleri backend'den yeniden üretir → tip kontrolü → testler → üretim derlemesi.
Gizli bilgi taraması	Anahtar/özel anahtar desenleri için çalışma ağacı **ve tüm commit geçmişi**; `.env` takip ediliyorsa kırmızı.

Test felsefesi

- **Her önemli hata için, davranışı adlandıran bir regresyon testi.** Ör.: `test_rag_abstention` (alakasız soruda sahte sağlayıcının çağrı sayacı artmaz), `test_space_scope` (iki erişim yolu), `test_prompt_injection` (saldırılar işaretlenir, *sıradan ders metni işaretlenmez* — her şeyi işaretleyen bir ölçü aletine kimse bakmaz), `test_service_budget`, `test_rate_limiting`, `test_oauth_flow`.
- **Sahte bağımlılıklar gerçek sözleşmeyi taklit eder.** Frontend'de sahte `EventSource`, tarayıcı sözleşmesinin iki ayrıntısını yansıtır (kapalı kaynak hiçbir şey iletmez; kopan bağlantı bodysiz bir `Event`'tir, `MessageEvent` değil): sıkıntılı bir sahte, gerçekte olamayacak davranışı doğrulardı.
- **Test edilmesi gereken yerde mock'tan kaçınmak.** PKCE hatası, hem yetkilendirme URL'i hem token değişimi mock'landığı için testlerden kaçmıştı; sonradan gerçek devreyi çalıştıran testler eklendi.
- **Tümü yapılandırma paritesini de kilitler.** `test_env_example_parity` her yapılandırma anahtarının `.env.example`'da olduğunu, `test_compose_topology` Compose tarifinin tutarlılığını, `test_dialect_translation` SQLite → Postgres çevirisini doğrular.

Ölçüm araçları (ürünle birlikte gelen)

Betik	Ne ölçer
`scripts/measure_ranking.py`	Saklı çalıştırmalarda sıralama etkileri; `POPULARITY_GATE_FULL` eşik taraması. (Kısa liste *içinde* yeniden sıralar, daha iyi bir video var mıydı sorusunu yanıtlamaz.)

Betik	Ne ölçer
`scripts/measure_rag_thresholds.py`	Etiketli sorular → benzerlik dağılımı, eşik taraması. **İki hatayı birden** basar: yanlış "bulamadım" ve gereksiz LLM çağrısı (yalnız birini sıfırlamak eşiği dibe çeker, kapının varlık nedeni ikincisidir).
`scripts/backfill_embeddings.py`	Eksik vektörleri listeler / üretir. Yeniden çalıştırması güvenli.
`scripts/dump_openapi.py`	OpenAPI + SSE gövdeleri → TypeScript tipleri.
`GET /api/admin/usage`	Canlı: kota, olaylar, soğumalar. Eşzamanlılık ayarı (`MAX_*_WORKERS=4`) bu sayılara bakılarak değiştirilecek, tahminle değil.

BÖLÜM 10

Dağıtım, ölçekleme ve bilinen sınırlar

Neyin hazır olduğu, neyin bilerek ertelendiği, neyin gerçek sınır olduğu.

Ne var, ne yok

Hazır

- Çok aşamalı Docker imajı, kök olmayan kullanıcı, sağlık kontrolü.
- Compose: web + worker + Redis (kalıcı) + Postgres (sağlık kontrollü).
- SQLite → Postgres veri aktarımı (`python -m src.storage.transfer` , `--dry-run` , dolu hedefi reddeder).
- Google OAuth girişi, oturumlar, kullanıcı başına ve servis geneli kota tavanları.
- Yönetici kullanım raporu, sır maskeleme, sahipsiz veri temizliği.

Bilerek ertelenen (bugün yapmak yanlış olurdu)

- İstek hızı sınırlayıcı middleware: gerçek eşzamanlılık verisi yokken tahminle ayar olurdu; günlük kota şimdilik yerini tutuyor.
- Nesne depolama: yüklenen dokümanlar hâlâ dosya sisteminde; web ve worker **aynı birimi** paylaşmalı.
- Çok turlu sohbet hafızası, cross-encoder yeniden sıralama, alanlar arası paylaşım (yeni yetki modeli), taranmış PDF için OCR.

Gerçek sınırlar (sunumda dürüstçe söylenebilecekler)

Sınır	Ayrıntı ve hafifletme
YouTube kotası	Tüm kullanıcılar toplamı için günde ~8 çalıştırma (Pasifik saatiyle sıfırlanır). Süreç sayısı artırmak bunu **yükseltmez**; ölçek RAG sohbeti, ingest, ASR ve erişilebilirliği kazanır. Gerçek çözüm: Google Cloud'da kota artışı talep edip `YOUTUBE_DAILY_QUOTA_UNITS` 'i güncellemek (kod değişikliği gerekmez).
Resmi olmayan yollar	yt-dlp ve transkript API'si, YouTube değişikliklerini yakından izler ve IP başına hız sınırına takılabilir. Resmi Data API birincil; yt-dlp yedek. `YTDLP_COOKIES_FROM_BROWSER` sınırları gevşetir. Transkript yoksa playlist yine kurulur (metadata sıralaması).
Tek süreç modu	Varsayılanda `--workers` yasak; iş durumu bellekte. Redis + Postgres ile kalkar. SQLite yalnız *tek makinede* çoklu süreç destekler (ağ dosya sistemlerinde WAL kilitleme güvenilir değil).
Dile bağlı doğrulama	Çapraz dilde kapı 3b ölçemez; diğer katmanlar (ayraçlar, sistem talimatı, 3a) çalışır.
OCR yok	Taranmış PDF "kapsam dışı" olarak işaretlenir; arayüz bunu açıkça söyler.

Sınır	Ayrıntı ve hafifletme
Embedding modeline özgü eşik	0,57 yalnız `gemini-embedding-001` için ölçüldü. `BAAI/bge-m3` veya başka modelde yeniden ölçülmeli.
ASR maliyeti	CPU'da video başına 90–120 sn; çok kullanıcıda ayrı worker havuzu ve sıkı kota gerekir.
Zayıf eşleşme kaçınılmaz	Havuzda konuyu gerçekten kapsayan video yoksa pick zayıf kalır; arayüz bunu "zayıf eşleşme" olarak *alaka* üzerinden etiketler.

Canlı demo için pratik notlar

> **Hedef: "herkese açık ürün" değil, gösterilebilir canlı demo**
> Hedef, birkaç gerçek kullanıcının kullanabildiği bir canlı demo. Bu çıtada gizlilik politikası, hata izleme, yedekleme, yatay ölçek ve Google OAuth doğrulaması ertelenebilir. Tek süreç + SQLite bu çıtada sorun değil. Gerçek kısıtlar:
>
> - **YouTube kotası:** tüm kullanıcılar için ~6–8 çalıştırma/gün (`MAX_UNITS_PER_DAY=8000` gibi bir tavanla).
> - **Doğrulanmamış Google OAuth ekranı:** hassas `auth/youtube` kapsamı nedeniyle ~100 kullanıcı tavanı ve uyarı sayfası.
> - **Canlıda `AUTH_MODE=single_user`'a düşmeyin:** o modda her ziyaretçi `DEFAULT_USER_ID` olur ve sahibin YouTube hesabına playlist yayınlayabilir. `multi_user` + `SECRET_ENCRYPTION_KEY` + `MAX_RUNS_PER_USER_PER_DAY=3`.
> - **Defter özelliği varsayılan kapalı:** `ENABLE_RAG=true` gerekir.

Anahtar yapılandırma değişkenleri

Değişken	Varsayılan	Etkisi
`LLM_PROVIDER`	`gemini`	gemini | together | openrouter; yalnız seçilenin anahtarı okunur
`AUTH_MODE`	`single_user`	single_user | multi_user (Google girişi)
`JOB_BACKEND` / `REDIS_URL`	`memory`	redis ise iş ayrı worker'da koşar
`DATABASE_URL`	boş (SQLite)	doluysa Postgres
`ENABLE_RAG`	`false`	öğrenme alanı: soru başına bir embedding + bir üretim çağrısı
`ENABLE_ASR_FALLBACK`	`false`	Whisper yedeği; `MAX_ASR_VIDEOS_PER_RUN` ile sınırlı
`MAX_RUNS_PER_USER_PER_DAY`	`0` (sınırsız)	multi_user için 3 önerilir
`MAX_UNITS_PER_DAY`	proje kotası	servis geneli kota tavanı
`SECRET_ENCRYPTION_KEY`	yok	yoksa OAuth jetonları düz metin (açılışta uyarı)
`RAG_MIN_SIMILARITY`	`0.57`	Kapı 1 eşiği, modele özgü
`RAG_MIN_ANSWER_GROUNDING`	`0.15`	Kapı 3b eşiği, bilerek düşük
`ADMIN_USER_IDS`	boş	multi_user'da boş = yönetici raporu kimseye açık değil

BÖLÜM 11

Sunum rehberi

Anlatı akışı, beklenen sorular ve sunumdan önce kapatılacak birkaç şey.

Önerilen anlatı (yaklaşık 12 dakika)

Süre	Bölüm	İçerik
1 dk	**Problem**	"Bir konuyu YouTube'dan öğrenmek isteyen biri hem yolu bulamıyor hem izlediği şeyi geri bulamıyor."
2 dk	**Demo 1: yol**	Hazır bir çalıştırmayı aç: alt konular, her slot için seçilen video, gerekçe, güven puanı ve **zayıf eşleşme etiketi**. Yeni bir çalıştırma başlatıp canlı ilerlemeyi (SSE) göster ama sonucu beklemeden hazır olana geç.
3 dk	**Demo 2: defter**	Bir defterde atıflı bir soru sor, atıfa tıklayıp videonun tam saniyesine git. Sonra **alakasız bir soru** sor ve "bulamadım" cevabını göster. Bu, sunumun en güçlü anı.
2 dk	**Mimari**	Bölüm 3'teki katman şeması. İki cümle: iş mantığı framework'ten bağımsız, iş yürütme soyutlanmış.
2 dk	**Dört kapı**	Bölüm 5'teki akış + eşik grafiği. "Eşiği ölçtük ve ilk ölçümümüz yanlıştı."
1,5 dk	**Sorunlar ve dersler**	3 hikâye seç: B1 (sentetik ölçüm), C2/C3 (yarış, kota), C7 (mock'un kaçırdığı hata). Örüntü: varsayım → ölçüm → düzeltme + test.
0,5 dk	**Sınırlar ve sonraki adım**	Kota tavanı, OCR yok, çapraz dilli 3b sınırı; yol haritası.

> **Demo riski: canlı çalıştırma**
> Bir çalıştırma 4–40 sn sürer, ~1.224 kota birimi harcar ve günlük kotanın ~%12'sidir. Ağ, kota veya yt-dlp sınırı sunum anında takılırsa demo çöker. Hazır çalıştırmaları ve üç dolu defteri (yerel veritabanında 60 kayıtlı çalıştırma, 3 defter, 18 kaynak, 286 parça ve 286 vektör) ana demo yapın; canlı çalıştırmayı ek gösterim olarak tutun.

Muhtemel sorular ve dayanaklı cevaplar

Modelin halüsinasyon görmediğini garanti edebilir misiniz?

Hayır ve iddia da etmiyoruz. İddia şu: cevapsız bir sorunun cevaplanmasını yapısal olarak zorlaştıran dört bağımsız kapı var, eşikleri gerçek defterlerde ölçtük ve sınırları belgeledik. En bilinen sınır çapraz dilli cevaplarda 3b'nin ölçüm yapamaması. Onu saklamıyoruz.

Neden vektör veritabanı kullanmadınız?

Defter başına birkaç bin parça var; 2.500 × 3072 float32 ≈ 30 MB ve numpy kosinüsü milisaniyeler sürüyor. Vektör DB, olmayan bir ölçek sorununu çözerdi. Ölçek gelirse Postgres zaten var.

Neden kullanıcıdan API anahtarı istemekten vazgeçtiniz?

Önce BYOK yaptık ve uyguladık. Sonra, kullanıcıdan Google Cloud projesi açıp iki anahtar üretmesini istemenin giriş engeli olarak fazla yüksek olduğuna karar verdik. Bedeli sunucu sahibine geçti; bunu günlük kullanıcı ve servis kota tavanlarıyla yönetiyoruz.

Kaç kullanıcıya hizmet verebilir?
Sınır CPU değil YouTube kotası: tüm kullanıcılar toplamı günde ~8 playlist üretimi. Defter sohbeti, doküman ingest'i ve ASR bu kotaya bağlı değil ve Redis + Postgres ile ölçeklenir. Playlist üretimi için Google'dan kota artışı talep etmek gerekiyor.

Neden SSE, WebSocket değil?
Akış tek yönlü. SSE proxy'lerden sorunsuz geçer, otomatik yeniden bağlanır ve `Last-Event-ID` ile kopan bağlantı kaldığı yerden devam eder. Bu tercih, iki sekmenin olayları bölüşmesi hatasının çözümüyle (C1) pekişti.

Prompt injection'a karşı ne yaptınız?
Altı katman: sistem talimatı, istem düzeni (soru en sonda), ayraçlar, çıktı doğrulaması (3b), girdi kısıtı ve ölçüm. Daha önemlisi etki alanı dar: cevap yolunda araç, dış istek veya otomasyon yok ve cevap düz metin basılıyor. Tarayıcı bilerek engellemiyor; prompt injection anlatan bir videoyu sormaya kullanıcının hakkı var.

Neden SQLite? Üretimde yeter mi?
Varsayılan yol sıfır altyapı için SQLite. Çok makineli kurulum için Postgres hazır: tek "lehçe" katmanı, Compose tarifi ve veri aktarım aracı var. İki veritabanında da aynı testler koşuyor.

Sıralamayı nasıl doğruladınız?
Saklı 13 çalıştırma / 42 seçim üzerinde ölçtük. Popülerlik kapısı 3 seçimi değiştirdi, üçü de daha iyi sözcüksel eşleşmeye; sıfır sinyalli seçim 6/42'den 4/42'ye indi. Bu, kısa liste içi yeniden sıralamayı ölçer; daha iyi bir videonun havuzda olup olmadığını değil, bunu da söylüyoruz.

Yapay zekâ ile mi geliştirildi?
Commit geçmişinde Claude "ortak yazar" olarak görünüyor. Sorulursa açıkça söylenebilir; vurgulanacak nokta, kararların (BYOK'un geri alınması, eşiklerin yeniden ölçülmesi, kapıların tasarımı) ölçüme dayandığı ve her düzeltmenin testle kilitlendiği.

Sunumdan önce kapatılacaklar

- ☐ **Commit edilmemiş çalışma var:** `.env.example`, `api/main.py`, `src/jobs/worker.py`, `src/utils/logging_utils.py`, `tests/test_env_example_parity.py` değişik, `tests/test_logging_setup.py` takipsiz (kök logger yapılandırması, C12). Testler bu hâlde de yeşil (729 geçti).
- ☐ **Dal ana dala birleşmemiş:** `feat/react-migration`, `main`'in **74 commit** önünde, geride değil. Sunumda "ana sürüm" gösterilecekse birleştirmek veya PR açmak gerekir.
- ☐ **README sayıları eski:** "471 test / 359 backend / 47 frontend" yazıyor; gerçek: **729 geçti + 71 atlandı** backend, **67** frontend.
- ☐ **README sağlayıcı listesi eksik:** "Gemini *veya* Together.ai" yazıyor; kodda ve `.env.example`'da **OpenRouter** da var.
- ☐ **Marka adı tutarsızlığı:** depo ve `PRODUCT.md` "Make A Playlist", arayüz kabuğu "Derslik" (`shell.brand`). Sunumda hangisini kullanacağınıza karar verin.
- ☐ **Demo verisi:** yerel veritabanındaki üç dolu defteri hazır tutun; `ENABLE_RAG=true` ile başlatın (`.claude/launch.json` içindeki `api-rag-dev` yapılandırması bunu yapar).
- ☐ **Kota:** sunumdan önce `GET /api/admin/usage` ile o günün kalan kotasına bakın.

EK A

API uçları ve veri modeli

28 HTTP ucu, 16 tablo.

API uçları

Yöntem	Yol	Açıklama
POST	`/api/runs`	Çalıştırma başlat → **202** + `run_id` (anında döner, ~207 ms ölçüldü)
GET	`/api/runs`	Geçmiş (sayfalı, kullanıcıya özel)
GET	`/api/runs/{id}`	Sonuç (`PlaylistResult`)
GET	`/api/runs/{id}/status`	Durum (pending / running / done / failed / cancelled / interrupted)
GET	`/api/runs/{id}/events`	SSE ilerleme akışı (heartbeat, `Last-Event-ID`)
DELETE	`/api/runs/{id}`	Çalışanı iptal et / kaydı ve dosyaları sil
POST	`/api/runs/{id}/publish`	YouTube'a yayınla (üretimden ayrı)
GET	`/api/runs/{id}/export/{artifact}`	JSON / Markdown indir (kayıtlı sonuçtan yeniden üretilir)
POST	`/api/spaces` · GET liste · GET/DELETE `/{id}`	Defter oluştur / listele / oku / sil
POST	`/api/spaces/{id}/sources/run`	Bir çalıştırmanın videolarını defterin kaynağı yap (iş + SSE)
POST	`/api/spaces/{id}/sources/document`	Doküman yükle (multipart, 20 MB, okunurken sınırlanır)
DELETE	`/api/spaces/{id}/sources/{sid}`	Kaynağı kaldır
GET	`/api/spaces/{id}/sources/{sid}/text`	Kaynak metni (transkript okuyucu)
POST	`/api/spaces/{id}/sources/{sid}/transcribe`	Transkripti olmayan videoyu ASR ile yazıya dök
GET	`/api/spaces/{id}/jobs/{job}/events`	İçeri alma işinin SSE akışı
POST	`/api/spaces/{id}/ask`	Soru sor → `RagAnswer` (cevaplandı / gerekçeli ret; HTTP 200)
GET	`/api/auth/youtube/start` · `/callback` · `/status`	YouTube OAuth yönlendirme akışı (PKCE, tek kullanımlık state)
DELETE	`/api/auth/youtube`	YouTube bağlantısını kes
GET · POST	`/api/auth/me` · `/api/auth/logout`	Oturum durumu (oturumsuzken 401 değil `signed_in:false`) · çıkış
GET	`/api/config`	Yetenek keşfi (sır sızdırmaz; testle kilitli)
GET	`/api/admin/usage`	Kullanım raporu (yöneticilere)
GET	`/api/health`	Sağlık kontrolü

Veritabanı tabloları (16)

Tablo	Amaç
`run` · `run_subtopic` · `run_video`	Çalıştırma geçmişi, `user_id` ile kapsanır; kabul anında yazılır (kota rezervasyonu burada)
`search_cache` · `transcript_cache`	Sağlayıcı/sorgu/filtre başına arama önbelleği (6 sa) · transkript ve video başına hatalar (30 gün / 15 dk)
`provider_cooldown` · `provider_event`	Sunucu geneli soğumalar ve ardışık hata sayaçları · günlük hız sınırı/hata/soğuma olayları
`api_usage`	Kullanıcı × gün × uç nokta başına kota birimi (YouTube bütçesinin kaynağı)
`session` · `oauth_state` · `oauth_token`	Oturumlar (token SHA-256 özeti) · bekleyen OAuth durumları (PKCE verifier dâhil) · şifreli YouTube jetonları
`space` · `space_source`	Defter ve kaynakları; kaynak durumları: indexed, no_text, failed, …
`chunk` · `chunk_fts` · `chunk_embedding`	Metin parçaları (zaman/sayfa konumlu) · FTS5 sanal tablosu (Postgres'te tsvector) · float32 BLOB vektörler (model adıyla)

Dizin yapısı

Yol	İçerik
`api/`	FastAPI: `main.py`, `deps.py`, `schemas.py`, `sse.py`, `routers/` (runs, spaces, auth, config, admin) — ~2.200 satır
`src/config/`	Ortam → doğrulanmış `AppConfig` (sunucu / kullanıcı sırrı / çalıştırma seçeneği ayrımı)
`src/providers/`	Dış sistemler, modül başına bir tane (LLM, YouTube Data API, yt-dlp, transkript API, Whisper) + hata sınıfları — ~2.700 satır
`src/services/`	Boru hattı, sıralama, atama, transkript, RAG, parçalama, embedding, doküman ayrıştırma, temizlik — ~4.300 satır
`src/storage/`	`sqlite_store.py` (tek depo), `dialect.py`, `crypto.py`, `transfer.py`, `factory.py` — ~2.700 satır
`src/jobs/`	Runner protokolü, bellek içi ve Redis uygulamaları, worker, görev kaydı — ~1.200 satır
`web/src/`	`api/` (istemci, tipler, sözleşme), `features/` (run, history, learn, publish, auth), `hooks/` (SSE, RAG), `i18n/`, `styles.css` (2,7 bin satır) — ~5,1 bin satır TS/TSX
`tests/`	56 dosya, ~13,1 bin satır, 670 test fonksiyonu (parametrelerle 800 koşum)
`scripts/` · `docs/`	Ölçüm ve bakım betikleri · `rag-plan.md`, `react-migration-plan.md`

EK B

Proje zaman çizelgesi

Git geçmişinden (87 commit) çıkarılan gelişim sırası.

Ağustos 2025 · İlk sürüm (v1.0)
Depo, "LLM-AGENT-PlaylistMaker" adıyla açıldı.

Nisan 2026 · Yeniden yapılanma
Video playlist üretim servisi eklendi (`feat: Implement video playlist generation service`).

13–14 Ağustos 2026 · Faz 0–4
Transkript API 1.x uyumu, Türkçe metin işleme, sağlayıcı sağlığı. FastAPI katmanı, SSE ilerleme, OAuth yönlendirme akışı, React + Vite + TypeScript arayüzü; Streamlit emekliye ayrıldı. GitHub Actions CI kuruldu. Olay kuyruğu → eklemeli günlük (C1).

15–16 Ağustos · Çok kullanıcılı temel
Google ile giriş, oturumlar, BYOK anahtarları, OpenAPI sözleşme kontrolü (kayma derleme hatasına dönüşür), kullanıcı başına günlük sınır (C2), `interrupted` durumu (C4), popülerliğin alakaya bağlanması (A6), eşdeğer terimler (A7).

19–21 Ağustos · Paylaşımlı anahtara dönüş
Together.ai desteği, çalışma notları. **BYOK → paylaşımlı sunucu anahtarı**. Yönetici kullanım raporu, soğumanın sunucu geneline alınması (C6), sekiz backend hatasının tek seferde düzeltilmesi (C14), servis geneli kota tavanı ve rezervasyon (C3), kapasite dolmadan kullanıcıyı önceden uyarma.

24–30 Ağustos · Ölçeklenebilir topoloji
Redis iş yürütücüsü ve worker, Postgres lehçe katmanı ve veri aktarımı, FTS sorgu katmanı, Dockerfile ve Docker Compose, sürüm sabitleme.

5–8 Eylül · Öğrenme alanı (RAG) olgunlaşması
Defter düzeyi sorular, prompt injection savunmaları, Kapı 3b, eşik yeniden ölçümü (B1–B6), ölü kod temizliği, `PRODUCT.md`, "müze etiketi" tasarım sistemi.

11 Eylül · Arayüz yenileme
TR/EN çeviri altyapısı (eksik çeviri = derleme hatası), yeni kabuk ve giriş ekranı, düz metin cevaplar (B7).

Eylül sonu · Yayına hazırlık
Kök logger yapılandırması (C12); canlı demo hedefi.

> **Tek cümlelik özet**
>
> Make A Playlist, "yol çiz" ve "topladığına sor" işlerini aynı ürüne toplayan, kararlarını ölçümle veren, cevaplayamadığı yerde **sessiz kalmak yerine söyleyen** ve her düzeltmesini bir testle kilitleyen bir sistem.